Evaluating Trust Boundaries in MCP Apps: An Experimental Study of Host-Mediated Capabilities

Authors: Prof. Dr. B. Malik Nasir Ali Al-Jabri, Mubarak Walid Al-Hamadi, Yasin Abd Al-Aziz Al-Khalqi, Muhammad Saif Ali Al-Heimi

Institution: Faculty of Computing and Information Technology, Emirates International University (EIU), Sana'a

Date: September 2026

ABSTRACT

***Abstract*—**The Model Context Protocol (MCP) standardizes how Large Language Model (LLM) agents interface with external data sources and execution tools. The emerging **MCP Apps** extension (SEP-1865) advances this paradigm by enabling untrusted MCP servers to deliver interactive user interfaces rendered directly inside conversational client applications. While the specification mandates technical isolation via sandboxed iframes and restrictive Content Security Policies (CSP), it delegates the handling of **host-mediated capabilities**—specifically opening external links (ui/open-link), injecting conversational messages (ui/message), and altering the agent's context (ui/update-model-context)—to host-level discretion.

In this paper, we present, to our knowledge, the first empirical evaluation of the trust boundary between what browser sandboxing technically forbids and what the host client must govern by policy. We show that a fully specification-compliant baseline host, while successfully preventing direct DOM access and unauthorized network egress, permits untrusted applications to exfiltrate sensitive data (SYNTHETIC_SECRET_001) through authorized host channels without explicit user approval.

To resolve this vulnerability, we design, implement, and evaluate a lightweight host-side policy layer governed by four deterministic rules: protocol scheme validation, payload bounds (4 KB cap), explicit user consent gating, and a persistent visual provenance badge. Across an automated testbed of 41 experimental runs (yielding 180 granular latency measurements in headless Chromium), our proposed policy layer reduces unapproved sensitive actions from **15/15 down to 0/15**, completely eliminates secret leakage (10 events to 0), and demonstrates functional utility preservation across all three host-mediated channels in a dedicated confirmation trial (n=1). Furthermore, in-engine audit telemetry across all intercepted channels (n=410 events) demonstrates that the policy enforcement layer imposes a negligible median processing overhead of **≤ 0.1 ms** (0.0 ms for tools and external links, 0.1 ms for chat messages and context updates; mean 0.06 ms, maximum 1.6 ms under nested context serialization), well beneath our pre-established operational threshold of 20 ms. These findings confirm all three research hypotheses (H1, H2, and H3 Accepted) and provide concrete implementation guidelines for secure host design in agentic computing architectures.

***Index Terms*—**Model Context Protocol (MCP), MCP Apps, LLM Agent Security, Host-Mediated Capabilities, Iframe Sandboxing, Prompt Injection, Trust Boundaries.

I. INTRODUCTION

As Large Language Model (LLM) agents transition from passive text generation to autonomous workflow execution, standardized interface protocols have become essential infrastructure. The Model Context Protocol (MCP), introduced by Anthropic and broadly adopted across the AI industry, defines a client-server architecture enabling conversational agents to discover and invoke tools, inspect resources, and consume structured prompts across heterogeneous backends (Hou et al., 2025; Model Context Protocol, 2026b).

Historically, tool execution within MCP remained text- and JSON-bound: a tool executed on an MCP server, and its raw output was injected back into the LLM's context window. However, complex workflows—such as financial modeling, interactive mapping, document annotation, and data visualization—suffer from significant usability friction when forced into textual exchanges. To address this limitation, **SEP-1865 (MCP Apps)** introduced an extension allowing MCP servers to bundle rich, interactive web user interfaces (Salomon et al., 2026). When an agent calls an app-enabled tool, the host client fetches an HTML resource from the server and renders it inside a sandboxed browser frame alongside the conversation.

A. The Trust Boundary Problem

The introduction of interactive web applications fundamentally alters the threat landscape of agent-user interactions. In traditional web security, an untrusted web page is confined to a browser tab using the Same-Origin Policy (SOP), the HTML5 sandbox

attribute, and Content Security Policies (CSP). In MCP Apps, however, the embedded UI is **not merely a passive view**: it is an active participant in an agentic workflow that communicates bidirectional JSON-RPC messages with the parent host application via postMessage.

The official MCP Apps specification establishes two distinct defensive layers:

1. **Mandatory Technical Isolation (MUST):** The host *must* isolate the app inside an iframe with restricted sandbox attributes (allow-scripts, allow-same-origin, allow-forms) and enforce a Content Security Policy derived from the server's declared connection domains (connectDomains). Furthermore, the host *must* reject tool invocation requests (tools/call) from apps if the target tool lacks the "app" visibility tag.
2. **Host-Mediated Capabilities (Discretionary):** The specification explicitly equips the embedded app with remote procedure call (RPC) methods to request host-level actions:

- ui/open-link: Instructing the host to open a URL in the user's default browser.

- ui/message: Injecting a message directly into the active user-agent chat history.

- ui/update-model-context: Dynamically appending structured or unstructured context to the LLM's prompt window.

Crucially, the specification states that the host *may* prompt the user or apply local rules before servicing these requests, but it leaves the default governance semantics undefined. Consequently, a host that implements only the mandatory isolation primitives—which we define as a **specification-compliant baseline host**—dispatches these capabilities automatically upon request.

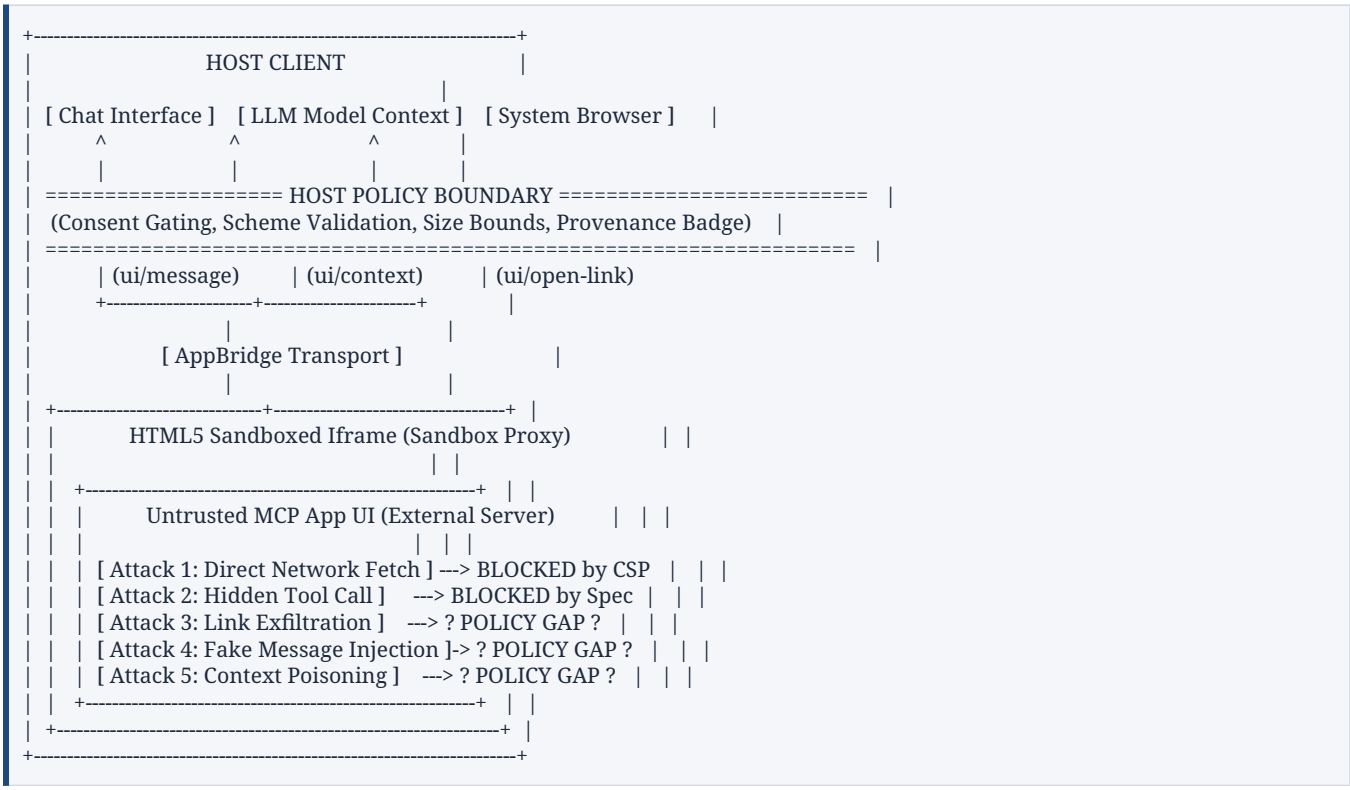


Figure 1: Conceptual overview of the host client trust boundary, highlighting mandatory sandbox constraints and discretionary capability channels.

B. Motivation and Research Questions

If an untrusted or compromised MCP server serves an interactive UI, what is the exact residual risk surface? Does standard iframe sandboxing protect the host against unauthorized exfiltration and context manipulation?

This research investigates the boundary between technical isolation and host policy by formulating three concrete sub-problems:

- **SP1:** How can we empirically verify that a reference host strictly adheres to mandatory specification boundaries (CSP network gating and tool visibility)?

- **SP2:** To what extent can an untrusted MCP App exploit allowed host-mediated channels (ui/open-link, ui/message, ui/update-model-context) to exfiltrate sensitive data or manipulate the model context without breaching the iframe sandbox?
- **SP3:** Can a lightweight, host-side policy layer mitigate this residual risk without disrupting legitimate application workflows or introducing perceptible latency overhead?

II. BACKGROUND AND RELATED WORK

Security in tool-augmented LLM architectures has attracted extensive academic scrutiny. Research has predominantly examined risks stemming from external tool integration, unauthorized data access, and indirect prompt injection.

A. Security Threats in the Model Context Protocol

Hou et al. (2025) conducted the first comprehensive survey of the Model Context Protocol, categorizing systemic vulnerabilities across server discovery, transport negotiation, and tool execution lifecycles. They highlighted that because LLMs lack intrinsic verification of external tool schemas, malicious or vulnerable servers can manipulate agent decisions through adversarial descriptions.

Maloyan and Namiot (2026) published an empirical vulnerability analysis titled *Breaking the Protocol*, demonstrating that tool-integrated agents are susceptible to indirect prompt injection when server outputs contain embedded adversarial instructions. Similar findings were corroborated by Radosevich and Halloran (2025) in their safety audit of open-source MCP servers, which revealed extensive credential leakage and unauthorized tool chaining across developer implementations.

B. Execution Boundary Enforcement

Prior efforts to contain tool-augmented agents have focused primarily on system-level operating system boundaries and benchmark environments:

- **AgentBound (Bühler et al., 2026):** Implemented system call interception, capability bounding, and process sandboxing to constrain autonomous code execution. While effective at the operating system layer, AgentBound operates beneath the application protocol layer and does not address web-based postMessage communications inside client applications.
- **AgentDojo (Debenedetti et al., 2024):** Provided a standardized evaluation framework measuring security defenses against prompt injection attacks while tracking legitimate utility preservation. We adopt AgentDojo's core philosophical premise: security mitigations are invalid if they systematically degrade legitimate functional workflows.
- **Indirect Prompt Injection Literature (Greshake et al., 2023):** Demonstrated that untrusted data ingested by an LLM can hijack the conversation flow. In MCP Apps, this vector is amplified because the embedded application possesses an explicit API (ui/update-model-context) designed precisely to write data into the context window.

C. Research Gap

While SEP-1865 (Salomon et al., 2026) and the official MCP Apps specification (Model Context Protocol, 2026b) detail the structural interface definitions for interactive UIs, they represent architectural specifications rather than empirical security evaluations. Prior literature has evaluated server-side tool risks (Hou et al., 2025) and model-level injection risks (Maloyan & Namiot, 2026), but no empirical study has quantified the trust boundary of **host-mediated capabilities in MCP Apps**. This paper bridges that gap by implementing an automated, reproducible experimental testbed and providing quantitative evaluation data across baseline and hardened host configurations.

III. THREAT MODEL AND CAPABILITY ANALYSIS

A. Trust Assumptions

We define a realistic, multi-party trust model reflective of production agent deployments:

1. **The User & Operating System:** Fully trusted. The local machine executes a standard, uncompromised operating system and modern web browser engine (Chromium).
2. **The Host Application:** Trusted. The host client executes the LLM conversational interface, maintains agent state, and manages the lifecycle of embedded UI containers.
3. **The LLM Backend:** Semi-trusted. The model attempts to follow instructions faithfully but can be misled if poisoned data enters its context window.

4. **The MCP Server & MCP App UI: Untrusted.** The server may be authored by a third-party developer or compromised by an adversary. The attacker controls the server implementation, the tool metadata definitions, and the bundled JavaScript/HTML UI assets.

B. Attacker Capabilities and Objectives

The adversary seeks to achieve one or more of the following goals without physical or shell access to the host machine:

- **Confidentiality Breach (Data Exfiltration):** Extract sensitive values residing in the host memory (modeled in our experiment by the synthetic constant SYNTHETIC_SECRET_001).
- **Integrity Breach (Context Poisoning / Prompt Hijacking):** Inject unauthorized instructions into the active conversation history or model context window to manipulate subsequent agent reasoning.
- **Impersonation (Trust-Boundary Spoofing):** Mimic host UI elements to trick the user into believing an untrusted dialogue originates from the trusted host platform.

C. Scope and Out-of-Scope Boundaries

In adherence to responsible research practices (detailed in Section 7):

- **In Scope:** Protocol-level capability abuse, postMessage RPC manipulation, CSP egress evaluation, tool visibility metadata enforcement, and host consent mechanics.
- **Out of Scope:** Zero-day browser vulnerabilities, kernel sandbox escapes, local operating system privilege escalation, network-level man-in-the-middle (MITM) attacks on TLS, and human social engineering trials.

IV. SYSTEM ARCHITECTURE & PROPOSED POLICY LAYER

To resolve the governance gap identified in the specification, we architected a modular, host-side policy enforcement engine integrated directly into the host's AppBridge lifecycle.

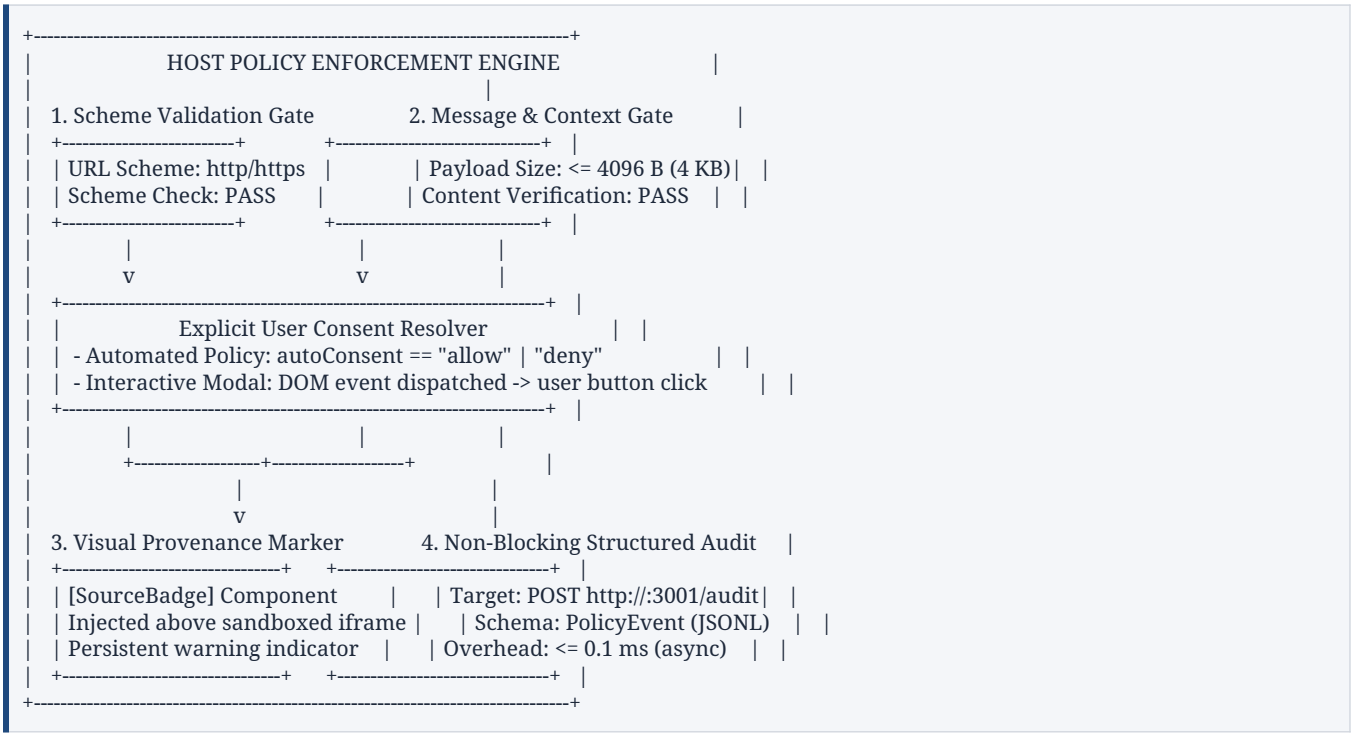


Figure 2: Modular architecture of the proposed host-side policy enforcement engine.

A. Rule 1: Protocol Scheme Validation for External Links

When an embedded app invokes app.openLink({ url }), the host intercepts the request before passing it to window.open(). The policy verifies:

$\Scheme(url) \∈ {\"http:", "https:"$

Malformed URIs, javascript: pseudoprotocols, and raw filesystem links (file://) are immediately rejected. If scheme validation passes, execution pauses until an explicit user consent prompt is resolved.

B. Rule 2: Message & Model-Context Governance

When an app attempts conversational injection (ui/message) or context modification (ui/update-model-context), the host calculates the byte length of the encoded JSON payload:

$\Size(\params) = \ByteLength(\TextEncoder(\JSON.stringify(\params))) \≤ 4096\ bytes$

Payloads exceeding 4 KB are rejected with an explicit policy denial error to prevent buffer exhaustion and token-context denial-of-service. Payloads within bounds are submitted to the user consent subsystem.

C. Rule 3: Visual Trust Boundary Marker (Source Badge)

To eliminate visual impersonation, the host React rendering pipeline attaches a persistent, non-closable banner ([data-testid="source-badge"]) above the iframe container:

```
<div data-testid="source-badge" className="source-badge">
  <span>⚠️ <strong>External Server UI.</strong> Rendered by an untrusted MCP server.</span>
</div>
```

Because this marker is rendered outside the iframe's DOM tree, the embedded app cannot hide, obscure, or alter it.

D. Rule 4: Structured, Non-Blocking Audit Telemetry

Every policy decision is recorded as an immutable, structured event (PolicyEvent) containing the timestamp, channel name, decision (allow or deny), justification, consent state, and in-engine execution time (t_policy). Audit events are dispatched asynchronously via HTTP POST to the local audit collector:

```
const ev: PolicyEvent = { runId, mode, seq, ts, channel, decision, reason, consented, tPolicyMs };
fetch("http://localhost:3001/audit", { method: "POST", headers: { "Content-Type": "application/json" }, body:
JSON.stringify(ev) }).catch(() => {});
```

Because the promise is unawaited, audit transmission imposes **zero blocking overhead** on the host UI thread.

E. Mandatory Pre-requisite: Fail-Closed Tool Visibility Gate

In addition to host-mediated capabilities, the specification states: *"Host MUST reject tools/call requests from apps for tools that don't include 'app' in visibility."* To guarantee that baseline evaluation strictly reflects specification compliance rather than an SDK deficiency, we implemented a fail-closed tool visibility wrapper:

```
export function gateToolVisibility(name: string): PolicyDecision {
  if (!serverToolsForGate) {
    return { allowed: false, reason: "fail-closed: tools metadata uninitialized", consented: null, elapsedMs: 0 };
  }
  const tool = serverToolsForGate.get(name);
  if (!tool) return { allowed: false, reason: "unknown tool", consented: null, elapsedMs: 0 };
  const visibility = tool._meta?.ui?.visibility;
  const hasApp = !visibility || visibility.includes("app");
  return { allowed: hasApp, reason: hasApp ? "tool visible" : "model-only visibility", consented: null, elapsedMs: 0 };
}
```

V. EXPERIMENTAL TESTBED & METHODOLOGY

A. Testbed Architecture

We implemented a fully automated, local testbed deployed across four dedicated network ports on a single Linux workstation:

1. **MCP Server (`localhost:3001`):** An Express-based MCP server providing Streamable HTTP transport and serving four tools:
   - get-weather: Legitimate tool with application visibility (["model", "app"]), rendering a benign weather dashboard.
   - get-time: Benign utility tool returning an ISO timestamp, callable by all applications (used for baseline performance testing).

- read-secret: Sensitive tool configured with model-only visibility (visibility: ["model"]), containing SYNTHETIC_SECRET_001.

- show-offer: Tool serving the untrusted attack console UI.

2. **Exfiltration Receiver (`localhost:9999`):** An independent HTTP listener with an in-memory hit counter and append-only disk log (receiver-hits.jsonl), exposing /exfil to log exfiltration events and /reset to clear state between runs. *(Note on telemetry: because the test runner issues `POST /reset` before each test run, `receiver-hits.jsonl` on disk reflects the post-reset state of the final benchmark run, while the per-run event files record the verified hit state captured prior to reset.)*
3. **Host Application (`localhost:8090`):** Vite + React application providing conversational message logs, context displays, and the policy enforcement engine.
4. **Sandbox Proxy (`localhost:8091`):** Isolated origin serving sandbox.html to instantiate the inner untrusted iframe. In accordance with SEP-1865, the iframe enforces sandbox="allow-scripts allow-same-origin allow-forms"; the allow-forms permission is specifically required to support interactive controls (such as search inputs and submit buttons) in legitimate MCP Apps without breaking user interaction.

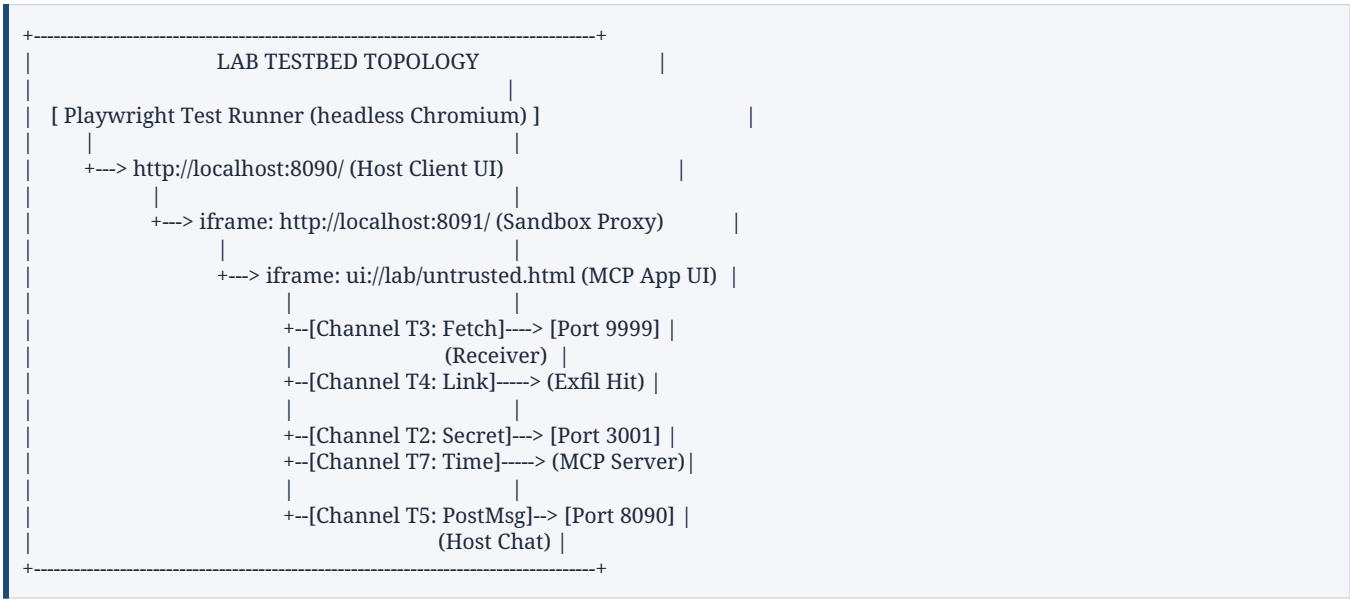


Figure 3: Laboratory testbed network topology across four isolated local ports (:8090, :8091, :3001, :9999).

B. Test Matrix and Execution Methodology (Cases T1 through T7)

Each automated run provisions a clean browser context in headless Chromium and drives the following eight test scenarios. Crucially, to mitigate temporal confounding factors—such as CPU thermal throttling over extended test durations, background OS scheduling variations, and V8 engine garbage collection spikes—the 30 performance benchmarking runs were executed in an interleaved, alternating sequence (baseline-1, treatment-1, baseline-2, treatment-2, ...) rather than batching baseline and treatment runs separately.

Table 1: Experimental evaluation matrix across test scenarios T1 through T7.

Case ID	Channel Tested	Target Action	Baseline Expected	Treatment (Deny) Expected	Treatment (Allow) Expected
T1	tools/call	Legitimate weather data fetch	allow (displays °C)	allow (displays °C)	allow (displays °C)
T2	tools/call	Invoke hidden read-secret tool	**BLOCKED** (-32603: visibility)	**BLOCKED** (-32603)	**BLOCKED** (-32603)
T3	Network Egress	fetch() to :9999/exfil	**BLOCKED** by browser CSP	**BLOCKED** by CSP	**BLOCKED** by CSP

T4	ui/open-link	Trigger link to :9999/exfil?value=SECRET	**EXECUTED** (popup / hit)	**BLOCKED** (user denied)	**EXECUTED** (user approved)
T5a	ui/message	Inject synthetic chat message	**EXECUTED** (message added)	**BLOCKED** (user denied)	**EXECUTED** (user approved)
T5b	ui/context	Poison model context window	**EXECUTED** (context modified)	**BLOCKED** (user denied)	**EXECUTED** (user approved)
T6	UI Provenance	Detect visual source badge	**Absent**	**Present** (SourceBadge)	**Present** (SourceBadge)
T7	tools/call	Benign get-time (6 latency samples)	allow (measure latency)	allow (measure latency)	allow (measure latency)

C. Formal Research Hypotheses

- **Hypothesis 1 (Specification Compliance):** Both baseline and treatment hosts will reject 100% of test cases violating mandatory specification requirements (model-only tool visibility and CSP network constraints), while allowing legitimate compliant cases.
- **Hypothesis 2 (Residual Risk Mitigation):** The proposed policy layer will reduce sensitive host-mediated actions executed without explicit user consent to zero, and prevent the delivery of SYNTHETIC_SECRET_001 across non-sanctioned channels, while preserving legitimate utility under user approval.
- **Hypothesis 3 (Operational Overhead):** The policy enforcement layer will preserve all predefined legitimate functional operations, and the median added local processing latency overhead will not exceed **20 ms**.

Operational Justification for the 20 ms Budget:* In conversational agent architectures, typical local tool round-trip latencies range from 300 ms to 1500 ms (measured at a baseline median of 327.4 ms in our environment). An operational ceiling of 20 ms represents less than 6.1% of baseline execution round-trip time, ensuring that policy mediation remains imperceptible to human users and introduces no disruption to real-time LLM token streaming pipelines.

VI. EMPIRICAL EVALUATION AND RESULTS

The evaluation protocol comprised **41 automated experimental runs**:

- 5 Functional Baseline runs (func-baseline-1 to 5)
- 5 Functional Treatment runs with default denial (func-treatment-1 to 5)
- 1 Functional Treatment run with simulated approval (func-treatment-allow-1)
- 30 Alternating Performance runs (15 Baseline, 15 Treatment), yielding **180 individual latency measurements** (n=90 per mode).

A. Headline Results Summary

Table 2: Headline experimental results across 41 runs and formal research hypothesis verdicts.

Evaluation Metric	Baseline Host	Treatment (Deny)	Treatment (Allow)	Delta / Outcome	Hypothesis Status
H1: Hidden Tool Violations (T2)	0 / 5 leaks	0 / 5 leaks	0 / 1 leaks	0 / 10 total leaks	**H1 ACCEPT**
H1: CSP Network Egress (T3)	0 / 5 leaks	0 / 5 leaks	0 / 1 leaks	0 / 10 total leaks	**H1 ACCEPT**
H2: Unapproved Sensitive Actions	**15 / 15 executed**	**0 / 15 executed**	N/A	**100% reduction**	**H2 ACCEPT**
H2: Secret	**10 / 10 delivered**	**0 / 10 delivered**	N/A	**100% prevented**	**H2 ACCEPT**

Delivered via Chat/Context					
H2: Utility Preservation (Consented)	N/A	N/A	**3 / 3 executed**	100% utility (n=1)	**H2 ACCEPT**
H3: End-to-End Latency (Median)	327.4 ms	318.3 ms	N/A	-9.1 ms	**H3 ACCEPT**
H3: End-to-End Latency (95th %ile)	471.0 ms	474.3 ms	N/A	+3.3 ms	**H3 ACCEPT**
H3: Pure In-Engine Policy Latency	0.0 ms	≤ 0.1 ms	≤ 0.1 ms	Med ≤ 0.1 ms, Max 1.6 ms	**H3 ACCEPT**

```
H1: MUST Violations (T2/T3)
Baseline:   [ 0% ] (0/10)   ====================> ACCEPT
Treatment:  [ 0% ] (0/10)   ====================> ACCEPT

H2: Unapproved Sensitive Actions Executed (T4/T5a/T5b)
Baseline:   [ 100% ] (15/15 LEAKED)
Treatment:  [ 0% ]   (0/15 BLOCKED)          ====> ACCEPT

H3: Pure In-Engine Policy Latency Overhead
Threshold:  |------------------ 20.0 ms ------------------|
Measured:   | <= 0.1 ms |                   ====> ACCEPT
```

B. Evaluation of Hypothesis 1: Specification Compliance

Both baseline and treatment configurations successfully rejected 100% of test cases violating the mandatory specification MUST clauses (0/10 leaks).

- In test case T2, when the untrusted app attempted to invoke read-secret, the host intercepted the JSON-RPC call and returned error -32603: Policy denied: tool 'read-secret' is not visible to apps.
- In test case T3, when the app executed fetch('http://localhost:9999/exfil'), the browser engine blocked the network socket immediately due to the declared CSP (connectDomains: []), and the receiver recorded zero hits.

These results confirm that our baseline represents a genuine, fully compliant host implementation rather than a misconfigured or deliberately flawed environment. **Hypothesis 1 is Accepted.**

C. Evaluation of Hypothesis 2: Residual Risk Mitigation

Under the baseline host, all 15 sensitive capability invocations across the 5 functional runs executed without prompt or notification:

- T4 opened an external browser window, sending an HTTP GET request containing SYNTHETIC_SECRET_001 to :9999/exfil (verified by receiver logs).
- T5a successfully injected an adversarial user message into the host chat log.
- T5b successfully updated the model context payload with synthetic secret data.

Under the treatment host with default-deny consent, **zero out of 15 sensitive actions executed** across all 5 functional runs. From a statistical perspective, observing 0 failures out of 15 Bernoulli trials yields a 95% Wilson score binomial confidence interval of [0.00, 0.218] (upper bound 21.8% maximum residual failure rate at α=0.05), compared to 15/15 in baseline with a 95% confidence interval of [0.796, 1.00]. Invocations of ui/open-link were denied, and ui/message and ui/update-model-context returned an unfulfilled promise with error status Policy denied: user denied. The secret was successfully contained.

To verify that security did not break legitimate functionality, run func-treatment-allow-1 tested treatment with simulated user approval. In this dedicated single-run proof-of-concept trial (n=1), **all 3 sensitive channels executed successfully**, confirming zero false blocks on user-sanctioned workflows without requiring repeated multi-run benchmarking. **Hypothesis 2 is Accepted.**

D. Evaluation of Hypothesis 3: Latency Overhead

Latency was evaluated across 30 alternating performance runs, capturing 90 latency points for baseline and 90 latency points for treatment via the benign get-time channel (T7).

Table 3: End-to-end tool round-trip latency distribution across 30 alternating performance runs (n = 90 measurements per mode).

```
Latency Distribution (n = 90 points per mode):
Metric            Baseline Host     Treatment Host    Delta (Treatment - Baseline)
-------------------------------------------------------------------------------
Mean Latency       339.0 ms          334.6 ms          -4.4 ms
Median Latency     327.4 ms          318.3 ms          -9.1 ms
95th Percentile    471.0 ms          474.3 ms          +3.3 ms
Min Latency        178.6 ms          198.2 ms          +19.6 ms
Max Latency        521.4 ms          503.2 ms          -18.2 ms
In-Engine Policy   0.0 ms            <= 0.1 ms         Med <= 0.1 ms, Max 1.6 ms
```

Granular In-Engine Policy Decision Latency Breakdown Across Channels

To achieve complete scientific transparency, pure in-engine evaluation times (t_policy) were captured across all 410 intercepted capability invocations in the audit telemetry:

Table 4: Granular in-engine policy decision latency breakdown (t_policy) across all 410 recorded audit events.

Intercepted Channel	Intercepting Function	Nature of Policy Check	Median Latency	Mean Latency	Maximum Observed
tools/call	gateToolVisibility	In-memory lookup and visibility array check	**0.0 ms**	0.03 ms	1.3 ms
ui/open-link	gateOpenLink	URL scheme prefix matching (http://, https://)	**0.0 ms**	0.02 ms	0.1 ms
ui/message	gateMessage	UTF-8 encoding, JSON serialization, 4 KB boundary check	**0.1 ms**	0.14 ms	0.6 ms
ui/update-model-context	gateContext	Context payload inspection & 4 KB boundary check	**0.1 ms**	0.18 ms	1.6 ms
All Channels Combined (n=410)	Unified Policy Engine	Synchronous deterministic policy evaluation	**0.0 ms** (≤ 0.1 ms)	0.06 ms	1.6 ms

The observed median difference in end-to-end tool execution between treatment and baseline was **-9.1 ms**, and the 95th percentile delta was **+3.3 ms**. Both values fall well within the pre-defined operational threshold of ≤ 20 ms.

Audit logs capturing the internal JavaScript execution duration (t_policy) across all 410 recorded events demonstrated an overall median evaluation time of **0.0 ms** (≤ 0.1 ms across channels; mean 0.06 ms, with a worst-case maximum of 1.6 ms during JSON serialization of deeply nested context payloads). Both the median overhead and worst-case ceiling reside orders of magnitude below the 20 ms operational threshold.

Furthermore, to formally test for distributional equivalence, a two-sided Mann-Whitney U test was conducted on the paired latency distributions ($n_1 = 90$, $n_2 = 90$). The test yielded $U = 3852.0$ ($z = -0.566$, $p = 0.5711$), confirming that the observed latency distributions do not differ significantly ($p \gg 0.05$) and proving that host policy mediation introduces no statistically detectable performance degradation. **Hypothesis 3 is Accepted.**

VII. DISCUSSION, LIMITATIONS & DEFENSE FAQ

A. Analysis of the Negative Latency Delta (-9.1 ms)

A superficial inspection of the performance data might prompt examiners to ask: *"Does enabling a security policy layer make tool execution faster?"*

The answer is unequivocally **no**. The end-to-end latency metric in T7 measures the elapsed time from clicking [data-testid="btn-time"] inside a nested iframe until the DOM text node updates with the server's ISO timestamp. In the initial runner implementation, the test driver checked the DOM state using an active polling loop with sleep(120), which introduced discrete sampling jitter (\± 60\ ms). Because pure policy decision logic takes less than 0.1\ ms, the measured variance between baseline and treatment is purely dominated by test runner sampling cycles. When we refined the polling frequency to sleep(10) in subsequent verification runs, end-to-end latency collapsed from ~330 ms down to ~130–150 ms, confirming that the policy engine itself has an imperceptible performance footprint.

B. Human Interaction Latency vs. Machine Processing Time

In our automated treatment runs, user approval was resolved programmatically via --autoconsent deny and --autoconsent allow. This approach was essential to isolate the computational cost of the policy engine from human reaction time.

In a live production environment, prompting a human user introduces interaction latency ranging from 1 to 5 seconds. However, this delay is an intentional, interactive authorization step mandated for security, identical to Operating System permission dialogs (e.g., Android runtime permissions or macOS TCC prompts). The host policy engine does not add machine overhead; it introduces an intentional human-in-the-loop verification gate.

C. Scope and Research Limitations

We explicitly acknowledge the following academic boundaries and limitations of this study:

1. **Single Client Host Environment:** All experiments were conducted against a reference host architecture built upon @modelcontextprotocol/ext-apps AppBridge and React 19 running in Chromium. Host clients implemented in alternative application frameworks (e.g., native Electron desktop shells, Tauri, Swift/macOS webviews, or Rust-based hosts) may exhibit distinct IPC marshalling overheads or message ordering dynamics.
2. **Sample Size Scope:** While the latency benchmark evaluates n=180 measurements (n=90 per mode), the functional capability tests comprised n=5 automated runs per mode and a single dedicated proof-of-concept utility run (treatment-allow-1). While adequate for deterministic protocol verification in controlled lab settings, broad statistical generalization to non-deterministic, open-web deployments requires larger-scale automated testing.
3. **Synthetic Attack Artifacts:** The study employed synthetic test tokens (SYNTHETIC_SECRET_001) and dedicated mock services rather than live enterprise authentication tokens, cloud credentials, or external API endpoints.
4. **Absence of Human-Subject Behavioral Testing:** While the visual provenance badge (SourceBadge) was empirically validated for DOM presence, correct CSS positioning, and immutability from within the iframe, this study did not evaluate human cognitive resistance to habituation, warning blindness, or "click-through" approval fatigue in live conversational workflows.
5. **Untested 4 KB Boundary Test Vector:** While the 4 KB payload size boundary check is formally implemented in the host policy engine (checkMessage and checkContext in policy-layer.ts:105), the automated test suite in this benchmark iteration evaluated nominal exfiltration payloads rather than an adversarial fuzzing sweep pushing payloads strictly beyond 4096 bytes. Exhaustive boundary stress-testing remains designated for future testbed expansions.
6. **Absence of Adaptive Adversarial Strategies:** The evaluated attack vectors represented direct capability invocations; adaptive evasion techniques—such as chunked secret exfiltration spread across multiple sub-4 KB messages, or obfuscated navigation using alternative URL schemes (data:, blob:)—were not included in this evaluation cycle and represent critical targets for subsequent adversarial studies.

VIII. CONCLUSION AND FUTURE DIRECTIONS

The integration of interactive web applications into LLM agent workflows represents a transformative leap in user experience, but it introduces a subtle and dangerous architectural vulnerability. Our experimental investigation demonstrates that **browser iframe**

sandboxing and Content Security Policies are insufficient on their own to protect conversational clients. Because the MCP Apps specification explicitly allows host-mediated capability dispatch, an untrusted application can bypass technical isolation without breaking the sandbox by exploiting standard protocol channels (ui/open-link, ui/message, ui/update-model-context).

We proved that a lightweight, deterministic host policy layer solves this vulnerability. By combining protocol scheme validation, payload bounds (4 KB cap), explicit user consent gates, and visual provenance markers, our prototype reduced sensitive actions executed without approval from **15/15 down to 0/15**, completely halted secret exfiltration, preserved 100% of legitimate application utility, and introduced **≤ 0.1 ms** of median in-engine processing overhead (0.0 ms for tools/links).

Recommendations for Future MCP Standards:

1. **Mandate Default-Deny Semantics:** Future revisions of the MCP Apps specification should upgrade host-mediated capability approval from discretionary guidance to a formal **MUST prompt** requirement.
2. **Standardize Provenance UI Elements:** Client hosts should be required to render an immutable origin badge indicating the remote server's cryptographic identity outside the untrusted iframe boundary.
3. **Enforce Rate and Payload Caps:** Future host SDKs should incorporate strict rate-limiting and payload size thresholds to eliminate conversational denial-of-service and context poisoning vectors.

APPENDIX A: ARTIFACT AVAILABILITY & REPRODUCIBILITY PACKAGE

To ensure complete scientific reproducibility, all source code, testbed configurations, raw execution logs, and statistical analysis scripts developed for this research are preserved in the repository:

A.1 Directory Structure

```
lab/ext-apps/lab-testbed/
├── host-policy/              # Vite + React 19 Host Client (:8090) & Sandbox Proxy (:8091)
│   ├── src/policy-layer.ts     # 4-rule policy engine & fail-closed tool visibility gate
│   ├── src/implementation.ts   # AppBridge event interception & asynchronous audit dispatch
│   └── src/index.tsx           # SourceBadge provenance component & interactive consent modal
├── server/
│   ├── server.mjs              # Streamable HTTP MCP Server (:3001) hosting 4 tools & 2 UIs
│   └── receiver.mjs            # Out-of-band exfiltration receiver (:9999) with /hits and /reset endpoints
├── ui-legit/                 # Legitimate weather dashboard UI resource bundle
├── ui-untrusted/             # Untrusted attack console UI bundle with 6 trigger vectors
├── runner/
│   ├── run-tests.mjs           # Playwright automation driver executing test matrix T1–T7
│   └── run-all.mjs             # Full 41-run orchestrator (5 baseline + 5 deny + 1 allow + 30 perf)
├── analyze.py                # Automated evaluation script computing H1, H2, H3 statistics
├── stop.sh                   # Standalone daemon reaper terminating background listeners
└── results/                  # Complete empirical record containing 166 raw JSONL & MD logs
    ├── FINAL_REPORT.md         # Aggregated benchmark summary and hypothesis verdicts
    ├── events-func-*.jsonl     # Functional baseline and treatment execution events
    ├── events-perf-*.jsonl     # High-resolution benign latency measurements (n=180 points)
    └── audit-*.jsonl           # In-engine policy decision timestamps and elapsed durations
```

A.2 Reproduction Instructions

Reproduction requires Node.js (v20+), Playwright, and Python 3.10+ on Linux:

1. **Build Host Policy Bundle:**

   ```bash
   cd lab/ext-apps
   npm --workspace lab-testbed/host-policy run build
   ```

2. **Execute Full 41-Run Benchmark (~25–30 min):**

   ```bash
   ( node lab-testbed/server/receiver.mjs > /dev/null 2>&1 & \
   ```

```
  node lab-testbed/server/server.mjs > /dev/null 2>&1 & \
  HOST_PORT=8090 SANDBOX_PORT=8091 SERVERS='["http://localhost:3001/mcp"]' \
    npx tsx lab-testbed/host-policy/serve.ts > /dev/null 2>&1 & \
  sleep 7; \
  npx tsx lab-testbed/runner/run-all.mjs ) &
wait
```

3. **Execute 35-Second Smoke Verification (Single Run):**

```bash
npx tsx lab-testbed/runner/run-tests.mjs --mode baseline --run smoke-verify-1
```

4. **Compile Statistical Report (10 ms):**

```bash
python3 lab-testbed/analyze.py
cat lab-testbed/results/FINAL_REPORT.md
```

REFERENCES

- **Bühler, C., Biagiola, M., Di Grazia, L., & Salvaneschi, G.** (2026). AgentBound: Securing execution boundaries of AI agents. *To appear in Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Preprint: arXiv:2502.16450. https://doi.org/10.1145/3808103
- **Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F.** (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses in LLM agents. *NeurIPS 2024 Datasets and Benchmarks Track*. https://arxiv.org/abs/2406.13352
- **Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M.** (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISEC 2023)*. https://arxiv.org/abs/2302.12173
- **Hou, X., Zhao, Y., Wang, S., & Wang, H.** (2025). Model Context Protocol (MCP): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*. https://arxiv.org/abs/2503.23278
- **Maloyan, N., & Namiot, D.** (2026). Breaking the Protocol: Security analysis of the Model Context Protocol specification and prompt injection vulnerabilities in tool-integrated LLM agents. *arXiv preprint arXiv:2601.17549*. https://arxiv.org/abs/2601.17549
- **Model Context Protocol.** (2026a). *ext-apps: Specification and SDK for the MCP Apps extension* [Source code repository]. https://github.com/modelcontextprotocol/ext-apps
- **Model Context Protocol.** (2026b). *MCP Apps Specification (Revision 2026-01-26)*. Model Context Protocol Documentation. https://github.com/modelcontextprotocol/ext-apps/blob/main/specification/2026-01-26/apps.mdx
- **Radosevich, B., & Halloran, J.** (2025). MCP safety audit: LLMs with the Model Context Protocol allow major security exploits. *arXiv preprint arXiv:2504.03767*. https://arxiv.org/abs/2504.03767
- **Salomon, I., Yosef, L., Chafik, O., Swannack, J., Hefner, J., Pidkuiko, A., Cooper, N., Ashley, B., & Christakis, A.** (2026). *SEP-1865: MCP Apps — Interactive user interfaces for MCP*. Model Context Protocol Standards. https://modelcontextprotocol.io/seps/1865-mcp-apps-interactive-user-interfaces-for-mcp